**Practicum No. 6** **Date:**

# Perform stack push pop operations

## I. Practical Significance

Simulating PUSH and POP operations using MATLAB helps students understand how stack-based memory operations work in computer architecture. PUSH and POP are fundamental instructions used in procedure calls, interrupt handling, expression evaluation, and runtime memory management.
This practicum strengthens understanding of the stack pointer (SP), LIFO behavior, memory indexing, and safe stack handling. By simulating these operations, learners gain insights into processor stack architecture and the role of the stack in assembly language programming, enabling them to apply these concepts in embedded systems, OS design, and compiler development.

## II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency.

- ***Use*** *the principles of computer architecture to optimize system performance*
  *(As stated in the Course Structure)*

a. ***Implement*** and analyse register-level data transfer operations in a processor architecture.

## III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO2. *Evaluate*** the performance of the processor functionalities.

## IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- ***Simulate*** PUSH and POP operations for ***the given*** data..

## V. Related ADO(s) *(As stated in the Course Structure)*

- ***Follow*** industry standards for system evaluation and performance optimization.
- ***Function*** effectively in teams to develop processor and memory architectures.
- ***Function*** responsibly in leadership roles to design computing systems.

## VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Pipelining is a fundamental technique in computer architecture used to improve instruction throughput by The stack is a fundamental data structure used in computer systems to support temporary storage of data during program execution. It operates on the Last-In-First-Out (LIFO) principle, meaning the most recently stored value is the first to be retrieved. In processor architecture, stack operations are handled using instructions such as PUSH and POP, which modify both the stack contents and the Stack Pointer (SP).
The Stack Pointer (SP) is a dedicated register that keeps track of the top of the stack. When a PUSH operation is performed, the SP is updated to point to the next available empty location, and the new data is placed on the stack. Conversely, during a POP operation, the data at the top of the stack is retrieved and removed, after which the SP is adjusted to point to the next valid element.
In traditional microprocessors such as the 8086, the stack grows downward in memory, while in some RISC architectures, the stack may grow upward. Regardless of growth direction, PUSH and POP operations ensure proper storage and retrieval of temporary data, return addresses, register values, and local variables during function calls and interrupts.

## VII. Algorithm *(Include probable photos of experimental set-up.)*

1. Start
2. Initialize MATLAB environment
3. Create an empty array to represent STACK
4. Initialize Stack Pointer (SP = 0)
5. Get user input or predefined data for PUSH operation
6. Perform PUSH operation:
   - Increment SP

- Insert data at STACK(SP)

7. Display stack contents after PUSH
8. Perform POP operation:
   - Read STACK(SP)
   - Decrement SP
9. Display stack contents after POP
10. Handle overflow and underflow conditions
11. Stop

## VIII. Resources Required

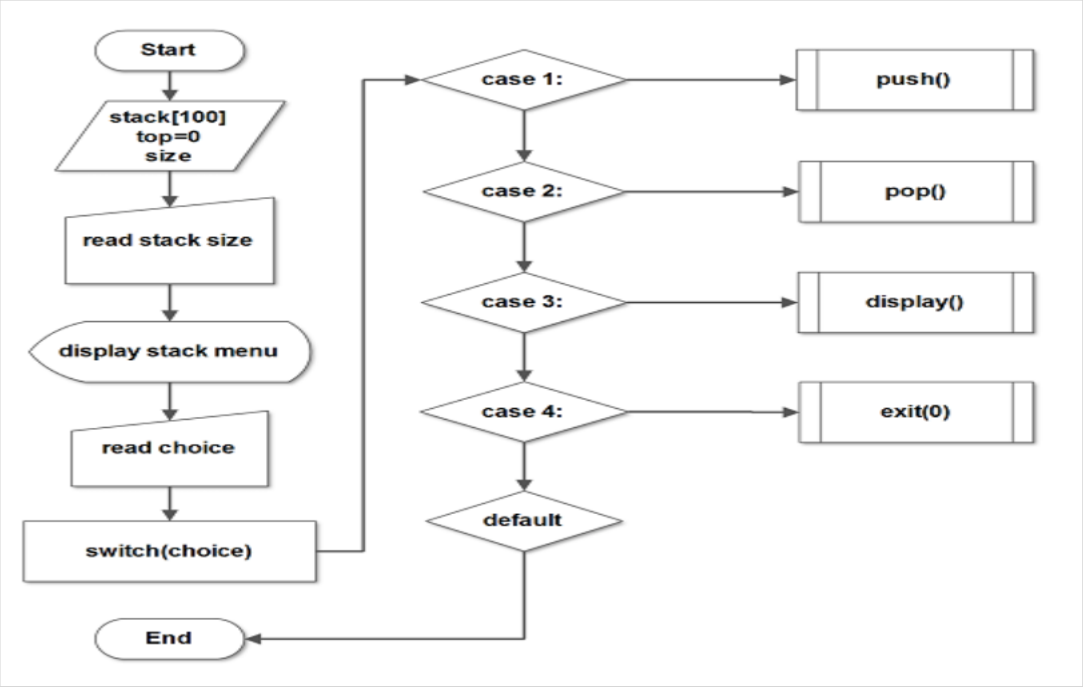
| S. No. | Resource | Suggested Broad Specification | Quantity |
|---|---|---|---|
| 1 | Laptop | Intel Core i5/i7 processor, 8GB RAM, Windows OS recommended for 8086 emulator | 1 No. |
| 2 | MATLAB | https://matlab.mathworks.com/ | 1 No. |

## IX. Safety Precautions

a) ***Use*** a comfortable keyboard and mouse position
b) ***Avoid*** unauthorized software downloads
c) ***Set*** screen brightness and contrast to comfortable levels
*d)* ***Adhere*** to the 20-20-20 rule to promote visual comfort and reduce eye strain
e) ***Avoid*** prolonged static postures
f) ***Organize*** your workspace properly

## X. Procedure *(Self-Instructional)*

### a) Flowchart




### b) Pseudocode

BEGIN

```
CLEAR all MATLAB variables
CREATE empty array STACK
SET SP = 0

DISPLAY "Enter value to PUSH"
READ value

// PUSH Operation
SP = SP + 1
STACK(SP) = value
DISPLAY "Stack after PUSH:", STACK

// POP Operation
popped_value = STACK(SP)
SP = SP - 1
DISPLAY "Popped value:", popped_value

DISPLAY "Stack after POP:", STACK

END
```

**c) Sample Matlab Code**

```
clc;
clear all;
close all;

disp('--- Simulation of PUSH and POP Operations ---');

% Initialize Stack
STACK = [];
SP = 0;    % Stack Pointer

% PUSH Operation
value = input('Enter a value to PUSH: ');

SP = SP + 1;      % Increase stack pointer
STACK(SP) = value;

disp('Stack after PUSH: ');
disp(STACK);

% POP Operation
if SP == 0
   disp('Stack Underflow! Cannot POP.');
else
   popped = STACK(SP);
   SP = SP - 1;
   disp(['Popped Value: ', num2str(popped)]);
end

disp('Stack after POP: ');
disp(STACK);
```

## XI. Actual Resources Used (*To be filled by the students*)

| S. No. | Resource | Suggested Broad Specification | Quantity |
|---|---|---|---|
| 1 | Desktop/Laptop | Intel i58GB RAM, MATLAB installed | 1 |
| 2 | MATLAB Software | MATLAB R2020a | 1 |
| 3 | Text Editor | MATLAB Editor | 1 |

## XII. Actual Procedure Followed

1. The computer system was switched on and MATLAB software was opened.
2. A new script file was created in the MATLAB editor and the stack simulation program was written.
3. An empty stack and stack pointer were initialized in the program.
4. A value was entered by the user to perform the PUSH operation and the stack pointer was incremented.
5. The POP operation was executed to remove the top element from the stack.
6. The stack contents before and after the operations were observed in the MATLAB command window.

## XIII. Observations

| S. No. | Observations | Data | Remarks |
|---|---|---|---|
| 1 | Stack initialized | STACK = [] | Empty stack |
| 2 | Value entered by user | Example: 25 | Value pushed to stack |
| 3 | PUSH operation executed | STACK = [25] | Stack pointer increased |
| 4 | POP operation executed | STACK becomes empty | Value removed from stack |

## XIV. Results

1. The MATLAB program successfully simulated PUSH operation, where a value was inserted into the stack and the stack pointer was incremented.
2. The POP operation removed the top element from the stack, demonstrating the LIFO (Last In First Out) principle used in stack data structures.

## XV. Interpretation of Results

The experiment demonstrates how stack operations work in computer systems. The PUSH operation adds a value to the top of the stack, while the POP operation removes the most recently added element. The stack pointer keeps track of the top element.
This simulation shows the **LIFO behavior of stacks**, which is commonly used in processor operations, function calls, and expression evaluation.

## XVI. Conclusions

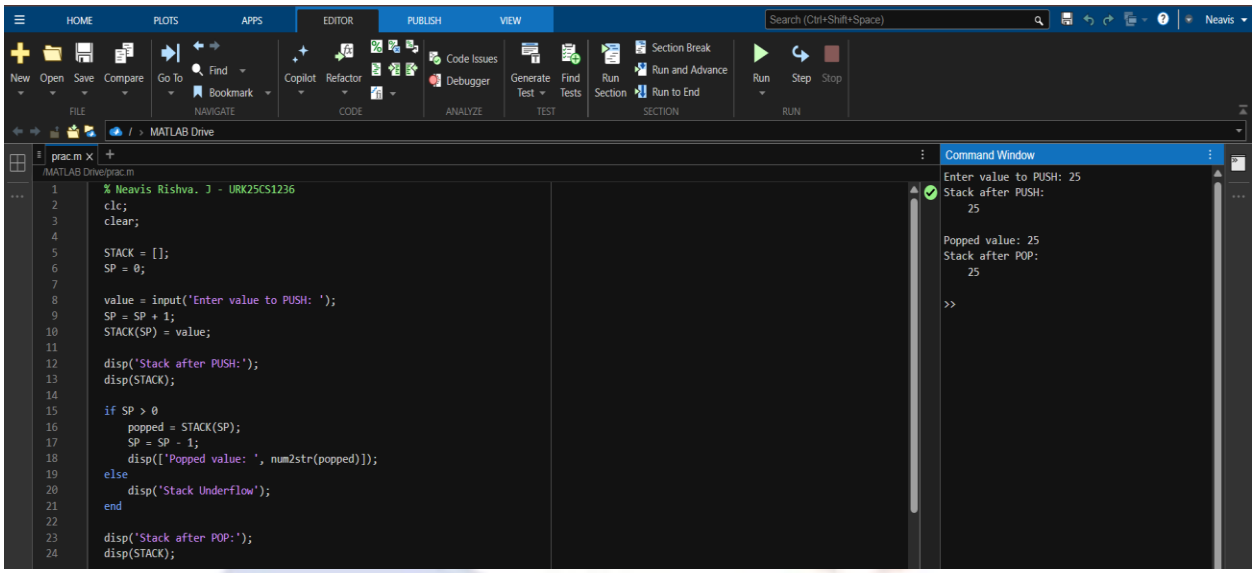
The MATLAB program successfully simulated **stack operations using PUSH and POP instructions**. The experiment confirmed that stacks follow the **Last In First Out (LIFO) principle**. This concept is widely used in computer architecture for managing **function calls, interrupts, and temporary data storage** during program execution.

## XVII. Suggested Practicum Related Questions:

***Note:*** *Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.*

1. Implement PUSH and POP operations using a vector as a stack in MATLAB.

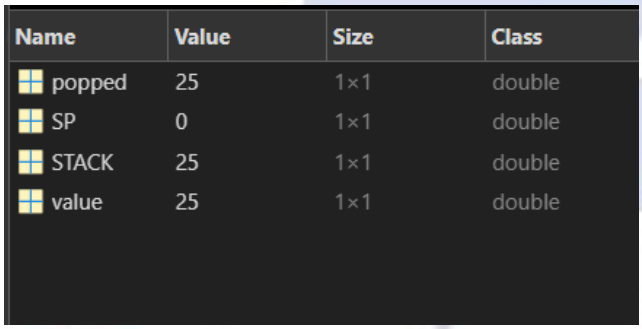
CODE:



```
% Neavis Rishva. J - URK25CS1236
clc;
clear;

STACK = [];
SP = 0;

value = input('Enter value to PUSH: ');
SP = SP + 1;
STACK(SP) = value;

disp('Stack after PUSH:');
disp(STACK);

if SP > 0
    popped = STACK(SP);
    SP = SP - 1;
    disp(['Popped value: ', num2str(popped)]);
else
    disp('Stack Underflow');
end

disp('Stack after POP:');
disp(STACK);
```

```
Enter value to PUSH: 25
Stack after PUSH:
    25

Popped value: 25
Stack after POP:
    25

>>
```

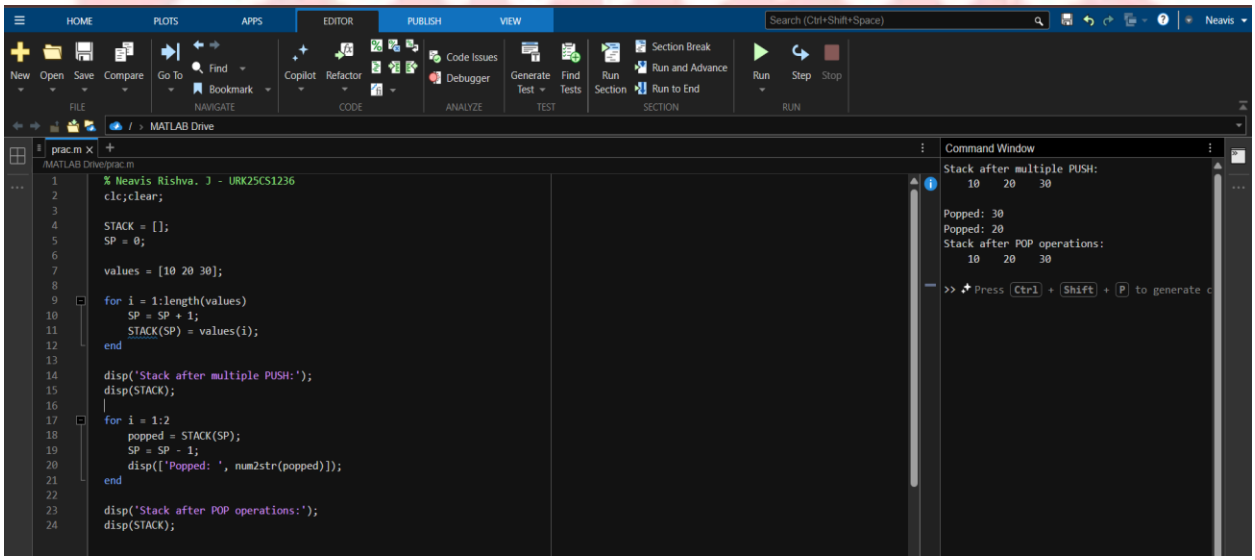
OUTPUT:



| Name | Value | Size | Class |
|---|---|---|---|
| popped | 25 | 1×1 | double |
| SP | 0 | 1×1 | double |
| STACK | 25 | 1×1 | double |
| value | 25 | 1×1 | double |

2. Simulate multiple PUSH and POP instructions in MATLAB using a sequence of operations.

CODE:



```
% Neavis Rishva. J - URK25CS1236
clc;clear;

STACK = [];
SP = 0;

values = [10 20 30];

for i = 1:length(values)
    SP = SP + 1;
    STACK(SP) = values(i);
end

disp('Stack after multiple PUSH:');
disp(STACK);

for i = 1:2
    popped = STACK(SP);
    SP = SP - 1;
    disp(['Popped: ', num2str(popped)]);
end

disp('Stack after POP operations:');
disp(STACK);
```

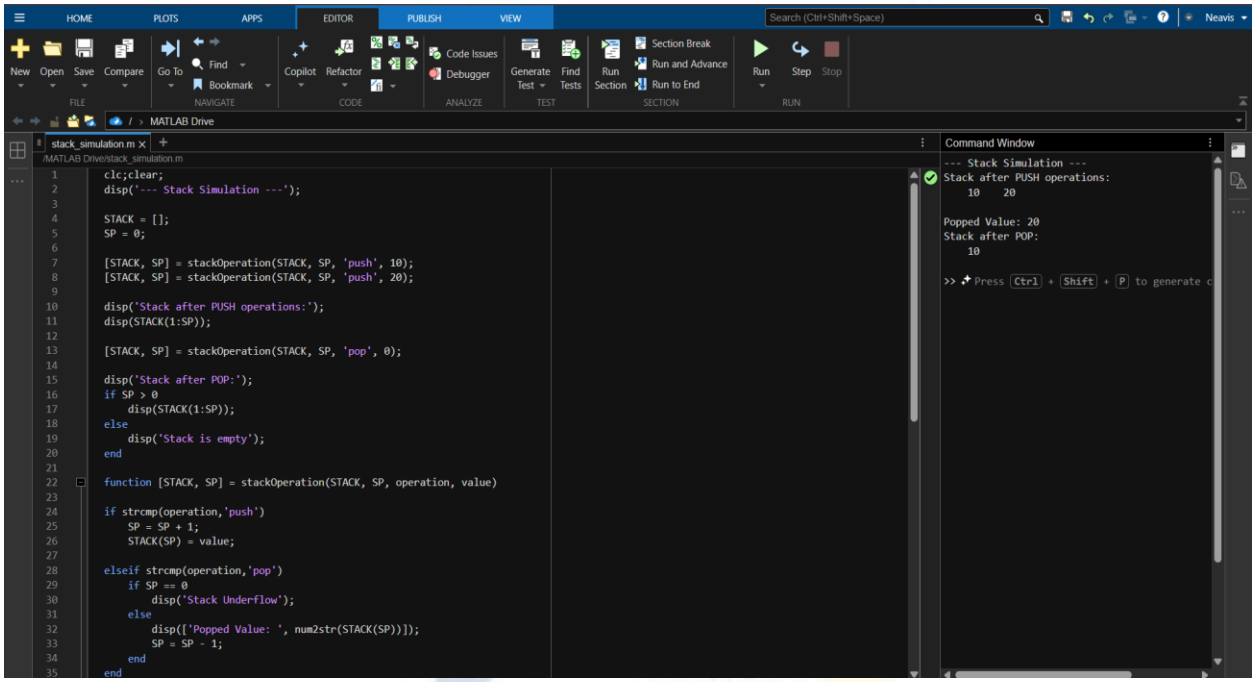
```
Stack after multiple PUSH:
    10    20    30

Popped: 30
Popped: 20
Stack after POP operations:
    10    20    30

>> Press Ctrl + Shift + P to generate c
```

OUTPUT:

| Name | Value | Size | Class |
|---|---|---|---|
| i | 2 | 1×1 | double |
| popped | 20 | 1×1 | double |
| SP | 1 | 1×1 | double |
| STACK | [10,20,30] | 1×3 | double |
| values | [10,20,30] | 1×3 | double |

3. Design a simple MATLAB function that returns updated stack and stack pointer values after each PUSH or POP.

CODE:



```
clc;clear;
disp('--- Stack Simulation ---');

STACK = [];
SP = 0;

[STACK, SP] = stackOperation(STACK, SP, 'push', 10);
[STACK, SP] = stackOperation(STACK, SP, 'push', 20);

disp('Stack after PUSH operations:');
disp(STACK(1:SP));

[STACK, SP] = stackOperation(STACK, SP, 'pop', 0);

disp('Stack after POP:');
if SP > 0
    disp(STACK(1:SP));
else
    disp('Stack is empty');
end

function [STACK, SP] = stackOperation(STACK, SP, operation, value)

if strcmp(operation,'push')
    SP = SP + 1;
    STACK(SP) = value;

elseif strcmp(operation,'pop')
    if SP == 0
        disp('Stack Underflow');
    else
        disp(['Popped Value: ', num2str(STACK(SP))]);
        SP = SP - 1;
    end
end
```

```
--- Stack Simulation ---
Stack after PUSH operations:
    10    20

Popped Value: 20
Stack after POP:
    10
```

OUTPUT:

| Name | Value | Size | Class |
|---|---|---|---|
| SP | 1 | 1×1 | double |
| STACK | [10,20] | 1×2 | double |

## XVIII. References / Suggestions for further reading


a) **8086 Assembly Language Programming – Stack Operations (Stanford University Course)**
https://cs.stanford.edu/courses/

b) **Khan Academy – Recursion and Stack Frames**
**https://www.khanacademy.org/computing/computer-science/algorithms/recursion**

c) **Intel 8086 Instruction Set Manual**
Official documentation describing PUSH, POP, and stack segment operations.
https://www.intel.com/content/www/us/en/content-details/671030/intel-64-and-ia-32-architectures-software-developer-manual-volume-2-instruction-set-reference-a-z.html

d) **MATLAB Documentation – Array and Memory Handling**
Explains MATLAB workspace, indexing, and array-based memory simulation.
https://www.mathworks.com/help/matlab/

e) **Neso Academy – Microprocessor Stack Operations (YouTube)**
Beginner-friendly explanation of PUSH, POP, SP, and stack behavior in 8086.
https://www.youtube.com/watch?v=3ecNf2S7fLA

## XIX. Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the *'process-related skills/LOs'* (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and *'product-related skills/LOs'*

**Note:** The weightage for the ***process-related skills*** and ***product-related skills*** will change depending on the PrO, COs, and the competency related to the concerned course.

| Performance Indicators | | Maximum Marks | Marks Obtained |
|---|---|---|---|
| **Process-related Skills: 18 Marks - 60%** | | | |
| 1 | ***Follow*** industry standards | 3 | |
| 2 | ***Function*** effectively in teams | 2 | |
| 3 | ***Function*** responsibly in leadership | 3 | |
| 4 | ***Simulate*** Push and Pop operation and identify different types of hazards. | 10 | |
| **Product-related Skills: 12 Marks - 40%** | | | |
| 5 | Uses correct commands or syntax in Tool | 3 | |
| 6 | Results | 3 | |
| 7 | Interpretation of Results | 3 | |
| 8 | Conclusion | 3 | |
| | **Total** | **30** | |

***To be filled by Student:***

| S.No. | Name of the Student | Register No. |
|---|---|---|
| 1 | **Neavis Rishva.J** | **URK25CS1236** |

***To be filled by the Faculty Member:***

| Marks Obtained | | | Name and Designation of the Faculty Member | Date of Evaluation |
|---|---|---|---|---|
| **Process-Related Skills (18)** | **Product-Related Skills (12)** | **Total (30)** | | |
| | | | | |

*******